THE AUTOWARE FOUNDATION

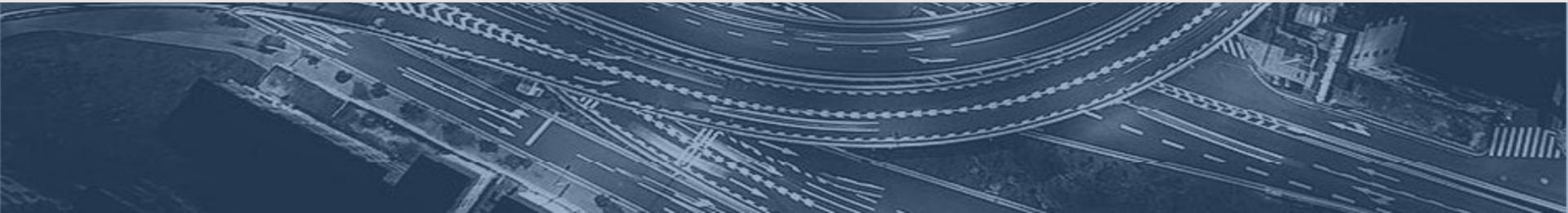
國立臺灣大學

NEWS Lab 嵌入式系統暨無線網路實驗室

nano-ros Safety Features

A Multi-Layered Approach to Safety-Critical Robotics

Hsiang-Jui (Jerry) Lin, National Taiwan University



Safety at Every Layer

- nano-ros applies defense-in-depth across the full stack.

Layer	Mechanism	Guarantee
Language	Rust ownership + borrowing	Memory safety without GC
Concurrency	async/await → state machines	Zero-overhead, known stack size
Compiler	#![no_std], no alloc, feature gates	No heap, no OS dependency
Types	heapless::Vec<T,N>, FixedSequence	Bounded memory at compile time
Serialization	CDR with bounds-checked writes	No buffer overflows
Scheduling	Deterministic executor, RTIC	Predictable timing
Networking	E2E CRC-32 + seq (EN 50159)	Corruption/loss detection
TSN	ThreadX + NetX Duo driver	Hardware-enforced determinism
Verification	67 Verus + 82 Kani proofs	Mathematical correctness

Rust Concurrency Without Overhead

- Rust's ownership system enforces thread safety at compile time — no runtime cost.

```
// Ownership prevents data races: only one mutable reference at a time
let mut msg = Odometry::default();
let ref1 = &mut msg;
// let ref2 = &mut msg;  // Compile error: cannot borrow twice

// Send + Sync traits — compiler checks thread-safety of every type
fn spawn_task<F: Send + 'static>(f: F) { /* ... */ }

spawn_task(move || {
    publisher.publish(&msg);  // Publisher must be Send — compiler verifies
});
```

- In C/C++, data races are runtime bugs. In Rust, they are compile errors.
- No mutexes for single-owner data. No thread sanitizer — the compiler is the sanitizer.

Async Without a Runtime

- Rust provides async/await syntax but no built-in runtime. The language gives you the abstraction; you choose the executor.

```
// This reads like threaded code...
async fn service_call(client: &mut Client<OperateMrm>)
    -> Result<Response, NodeError>
{
    let promise = client.call(&request)?;
    promise.await   // Suspends here, resumes when reply arrives
}

// ...but compiles to a state machine (no thread, no stack allocation):
enum ServiceCallFuture {
    WaitingForReply { client: ..., promise: ... },
    Done,
}
```

Async Without a Runtime — Poll + Waker

- The key insight: Promise implements Future using poll + waker.

```rust
impl<T> core::future::Future for Promise<'_, T> {
    type Output = Result<T, NodeError>;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>)
        -> Poll<Self::Output>
    {
        match self.get_mut().try_recv() {
            Ok(Some(reply)) => Poll::Ready(Ok(reply)),
            Ok(None) => {
                // Wake me when data arrives
                self.handle.register_waker(cx.waker());
                Poll::Pending
            }
            Err(e) => Poll::Ready(Err(e)),
        }
    }
}
```

- No busy-waiting. The waker notifies the executor only when data is ready.

Why Async Matters for Safety

- **Known size:** Stack size is known at compile time.
- Each async fn compiles to a state machine enum — the compiler computes its exact size. No per-task stack allocation.
- **Runs on a single-threaded MCU.** Multiple “concurrent” tasks share one thread:

```
// nano-ros executor: drives all async tasks on a single thread
pub async fn spin_async(&mut self) -> ! {
    loop {
        self.spin_once(1);   // Process all ready callbacks
        core::future::poll_fn::<(), _>(|cx| {
            cx.waker().wake_by_ref();
            Poll::Pending      // Yield to other tasks
        }).await;
    }
}
```

Why Async Matters for Safety — No select()

- Eliminates error-prone select()/poll() with fd sets. Rust async composes naturally:

```
// RTIC: two independent async tasks, zero shared state, zero locks
#[task(local = [executor], priority = 1)]
async fn net_poll(cx: net_poll::Context) {
    loop {
        cx.local.executor.spin_once(0);
        Mono::delay(10.millis()).await;
    }
}

#[task(local = [publisher], priority = 1)]
async fn publish(cx: publish::Context) {
    loop {
        cx.local.publisher.publish(&Int32 { data: 42 }).ok();
        Mono::delay(1000.millis()).await;
    }
}
```

- The compiler generates the same code as hand-written state machines, but with the readability of sequential code.

Compiler-Enforced no_std and Alloc-Free

- Every core crate compiles without the standard library. std and alloc are opt-in.

```
# nros-core/Cargo.toml
[features]
default = ["std"]
std = ["nros-serdes/std"]
alloc = ["nros-serdes/alloc"]
```

- Algorithm crates must cross-compile to bare-metal ARM:

```
cargo check --target thumbv7em-none-eabihf
```

- **If any code pulls in std or alloc, the compiler rejects it.**
- This is not a convention — it is a hard gate enforced on every CI run.

Compiler-Enforced Platform Selection

- Platform backends are mutually exclusive feature flags, not runtime configuration.

```
# nros-node/Cargo.toml — select exactly one
platform-posix      = [...]   # Linux, macOS
platform-zephyr     = [...]   # Zephyr RTOS
platform-freertos   = [...]   # FreeRTOS
platform-nuttx      = [...]   # NuttX
platform-threadx    = [...]   # Azure RTOS / ThreadX
platform-bare-metal = [...]   # No OS
```

- Same principle for RMW (middleware) backends:

```
rmw-zenoh = ["dep:nros-rmw-zenoh"]   # zenoh-pico (primary)
rmw-xrce  = ["dep:nros-rmw-xrce"]   # Micro-XRCE-DDS
rmw-dds   = ["dep:nros-rmw-dds"]     # Direct DDS
rmw-cffi  = ["dep:nros-rmw-cffi"]    # C FFI bridge
```

- **Wrong platform + wrong RMW = link error at build time, not runtime crash.**

Safe Types in Generated Message Code

- ROS 2 messages are code-generated into #![no_std] Rust crates. Dynamic containers are replaced with fixed-capacity alternatives.

```
// Generated from rcl_interfaces/srv/SetParameters
pub struct SetParametersRequest {
    pub parameters: heapless::Vec<Parameter, 64>,
}
```

- **Bounded:** heapless::Vec<T, 64> holds at most 64 elements — stack-allocated
- **Explicit overflow:** CapacityExceeded error, not silent truncation or panic
- **No allocator:** Works on bare-metal MCUs with zero heap

Safe Types — Deserialize with Capacity Check

- The generated Deserialize impl checks capacity on every push:

```rust
impl Deserialize for SetParametersRequest {
    fn deserialize(reader: &mut CdrReader) -> Result<Self, DeserError> {
        let len = reader.read_u32()? as usize;
        let mut vec = heapless::Vec::new();
        for _ in 0..len {
            vec.push(Deserialize::deserialize(reader)?)
                .map_err(|_| DeserError::CapacityExceeded)?;
        }
        Ok(Self { parameters: vec })
    }
}
```

- Overflow returns CapacityExceeded — not a panic, not silent truncation.

C++ Safe Types — Freestanding C++14

- The same fixed-capacity pattern applies to the C++ API. No STL, no exceptions, no RTTI.

```
// nros::FixedSequence<T, N> — C++ equivalent of heapless::Vec<T, N>
template <typename T, size_t N> struct FixedSequence {
    uint32_t size;
    T data[N];

    bool push_back(const T& val) {
        if (size >= N) return false;  // Capacity check — no heap
        data[size++] = val;
        return true;
    }
};
```

- Memory layout is repr(C) — identical on Rust and C++ sides, direct FFI.

C++ Safe Types — Error Handling

- NROS_TRY macro replaces try/catch — propagates errors via return codes:

```
nros::Result run() {
    nros::Node node;
    NROS_TRY(nros::create_node(node, "my_node"));

    nros::Publisher<std_msgs::msg::Int32> pub;
    NROS_TRY(node.create_publisher(pub, "/chatter"));

    std_msgs::msg::Int32 msg;
    msg.data = 42;
    NROS_TRY(pub.publish(msg));
    return nros::Result::success();
}
```

- No exceptions, no RTTI — works on freestanding C++14 targets.

Bounds-Checked CDR Serialization

- The CDR encoder checks every write against the buffer.

```
pub struct CdrWriter<'a> {
    buf: &'a mut [u8],    // Fixed buffer — no reallocation
    pos: usize,
    origin: usize,        // Payload origin for alignment
}
```

- Fixed-size buffer — no Vec, no reallocation, no heap.

CDR Serialization — Alignment Check

- Every write checks remaining capacity before touching the buffer:

```
pub fn align(&mut self, alignment: usize) -> Result<(), SerError> {
    let offset = self.pos - self.origin;
    let padding = (alignment - (offset % alignment)) % alignment;
    if self.remaining() < padding {
        return Err(SerError::BufferTooSmall); // Caught, not UB
    }
    for i in 0..padding {
        self.buf[self.pos + i] = 0;
    }
    self.pos += padding;
    Ok(())
}
```

- Buffer overflow is a compile-time-sized, runtime-checked error — never undefined behavior.

Deterministic Executor Scheduling

- The executor uses a fixed-size callback arena — no heap allocation during operation.

```
/// Type-erased callback metadata (no Box, no dyn Trait)
pub(crate) struct CallbackMeta {
    pub(crate) offset: usize,          // Position in arena
    pub(crate) kind: EntryKind,        // Sub / Srv / Timer / Action
    pub(crate) try_process:
        unsafe fn(*mut u8, u64) -> Result<bool, TransportError>,
    pub(crate) has_data:  unsafe fn(*const u8) -> bool,
    pub(crate) pre_sample: unsafe fn(*mut u8), // LET pre-sampling
    pub(crate) invocation: InvocationMode,
    pub(crate) drop_fn:    unsafe fn(*mut u8),
}
```

Deterministic Executor — Inline Entries

- Each callback entry is stored inline in the arena:

```rust
#[repr(C)]
pub(crate) struct SubInfoEntry<M, F, const RX_BUF: usize> {
    pub(crate) handle: RmwSubscriber,
    pub(crate) buffer: [u8; RX_BUF],  // Static receive buffer
    pub(crate) sampled_len: usize,
    pub(crate) callback: F,
    pub(crate) _phantom: PhantomData<M>,
}
```

- All sizes known at compile time. Arena capacity set via environment variables (NROS_EXECUTOR_MAX_CBS=64), enforced at build time.

RTIC Integration — Hardware-Scheduled Tasks

- On Cortex-M, nano-ros integrates with RTIC v2. No software scheduler — the NVIC enforces priorities.

```
#![no_std]
#![no_main]

#[rtic::app(device = stm32f4xx_hal::pac,
            dispatchers = [USART1, USART2])]
mod app {
    use nros::prelude::*;
    use std_msgs::msg::Int32;

    #[local]
    struct Local {
        executor: Executor,
        publisher: EmbeddedPublisher<Int32>,
    }
```

- All handles are #[local] — no locks required.

RTIC Integration — Async Task

- RTIC async tasks run on hardware priorities — no OS scheduler:

```
    #[task(local = [executor], priority = 1)]
    async fn net_poll(cx: net_poll::Context) {
        loop {
            cx.local.executor.spin_once(0);
            Mono::delay(10.millis()).await;
        }
    }

    #[task(local = [publisher], priority = 1)]
    async fn publish(cx: publish::Context) {
        let mut counter: i32 = 0;
        loop {
            counter = counter.wrapping_add(1);
            cx.local.publisher
                .publish(&Int32 { data: counter }).ok();
            Mono::delay(1000.millis()).await;
        }
    }
}
```

- RTIC guarantees data-race freedom through ownership, not mutexes.

E2E Safety Protocol (EN 50159)

- nano-ros treats the transport as an untrusted black channel.
- Enabled via a single feature flag:

Threat	Defense	Implementation
Corruption	CRC-32	Over CDR payload, in attachment
Repetition	Sequence number	Subscriber tracks expected seq
Deletion	Seq gap detection	Gap > 0 flagged immediately
Insertion	Seq + source GID	Rejects unexpected seq jumps
Resequencing	Monotonic seq	Non-monotonic = error
Delay	Timestamp	Stale messages rejected
Masquerade	Publisher GID	Random 16-byte identity

TSN — Feasible Path

- TSN requires hardware support. nano-ros's path: ThreadX + NetX Duo (TSN APIs included).
- **The ThreadX network driver is already implemented:**

```
// network.c — zenoh-pico transport over NetX Duo BSD sockets
#include "tx_api.h"
#include "nx_api.h"
#include "nxd_bsd.h"

z_result_t _z_open_tcp(_z_sys_net_socket_t *sock,
                       const _z_sys_net_endpoint_t rep, ...) {
    sock->_fd = nx_bsd_socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    struct nx_bsd_sockaddr_in addr;
    _z_ep_to_sockaddr(&rep, &addr);
    int rc = nx_bsd_connect(sock->_fd,
        (struct nx_bsd_sockaddr *)&addr, sizeof(addr));
    return (rc < 0) ? _Z_ERR_GENERIC : _Z_RES_OK;
}
```

TSN — Upgrade Path

- The upgrade path to TSN (NetX Duo already provides the APIs):
- The software stack is ready. The remaining step is hardware with a TSN-capable MAC.

Layer	Status	Details
nano-ros app	Working	zenoh-pico pub/sub + RPC
NetX Duo BSD sockets	Working	TCP + UDP unicast transport
NetX Duo TSN module	API available	CBS (802.1Qav), TAS (802.1Qbv), PTP (802.1AS)
TSN Ethernet MAC	Hardware upgrade	NXP i.MX RT1170/1180, Infineon AURIX TC3xx

Formal Verification — Verus Proofs

- Unbounded deductive proofs — mathematical guarantees for all inputs. 67 proofs across scheduling, CDR, E2E.

```
verus! {

/// OneShot timer fires exactly once, then becomes inert.
proof fn timer_oneshot_fires_once(s: TimerGhost, delta_ms: u64)
    requires s.mode is OneShot,
    ensures
        timer_fire_mode(s) is Inert,
        timer_fire_elapsed(s) == 0,
        !timer_update_ready(timer_after_fire(s), delta_ms),
{}

}
```

- Safety-critical one-time actions (e.g., emergency stop) can never repeat.

Verus Proofs — Drift-Free Scheduling

- Repeating timer preserves overshoot — no cumulative drift:

```
verus! {

/// Control loops fire at t=0, P, 2P, 3P... not t=0, P+e, 2P+2e...
proof fn timer_repeating_drift_free(s: TimerGhost, delta_ms: u64)
    requires
        s.mode is Repeating, s.period_ms > 0,
        sat_add(s.elapsed_ms, delta_ms) >= s.period_ms,
    ensures
        timer_fire_elapsed(TimerGhost {
            elapsed_ms: sat_add(s.elapsed_ms, delta_ms), ..s
        }) == sat_sub(
            sat_add(s.elapsed_ms, delta_ms), s.period_ms),
{}

}
```

- Excess time carries over to the next period — control loops stay aligned.

Formal Verification — Kani Model Checking

- Bounded model checking — exhaustive exploration of all execution paths. 82 harnesses across core crates.

```rust
#[cfg(kani)]
mod verification {
    use super::*;

    /// NaN velocity is treated as stopped (safe default).
    #[kani::proof]
    fn nan_velocity_treated_as_stopped() {
        let threshold: f64 = kani::any();
        kani::assume(threshold > 0.0 && threshold.is_finite());

        let filter = StopFilter::new(threshold, threshold, threshold);
        let result = filter.apply(&make_twist(f64::NAN, 0.0, 0.0));

        assert_eq!(result.linear.x, 0.0);  // NaN → stopped
        assert_eq!(result.linear.y, 0.0);
        assert_eq!(result.angular.z, 0.0);
    }
}
```

Kani — NaN Safety Pattern

- NaN comparisons require careful handling in safety-critical code:

```
// WRONG: NaN.abs() < threshold → false → NaN passes through
linear.x.abs() < self.vx_threshold

// RIGHT: !(NaN.abs() >= threshold) → !(false) → true → NaN treated as stopped
!(linear.x.abs() >= self.vx_threshold)
```

- Kani proves this holds for every possible f64 — including NaN, Inf, and subnormals.

Compile-Time Capacity via Environment Variables

- All buffer sizes and arena capacities are set at compile time — no runtime allocation decisions.

```
NROS_EXECUTOR_MAX_CBS=64              # Max callback slots
NROS_MAX_PARAMETERS=64                # Max ROS 2 parameters
NROS_PARAM_SERVICE_BUFFER_SIZE=8192   # CDR buffer per param service

ZPICO_MAX_PUBLISHERS=40               # zenoh-pico publisher slots
ZPICO_MAX_SUBSCRIBERS=16              # zenoh-pico subscriber slots
ZPICO_MAX_LIVELINESS=64               # zenoh-pico liveliness tokens
```

- Read by build.rs and baked into the binary as constants. Exceeding a limit returns an error — not a heap allocation.
- **Autoware Sentinel: 37 publishers, 9 subscribers, 1 timer, 11 services, 62 parameters — zero heap.**

Real-Time Lint Guide

- Clippy lints detect common real-time anti-patterns at build time:

```
cargo clippy -- \
    -D clippy::infinite_iter \
    -D clippy::while_immutable_condition \
    -D clippy::never_loop \
    -D clippy::empty_loop \
    -W clippy::large_stack_arrays \
    -W clippy::large_types_passed_by_value
```

- **Edition 2024 safety improvements:**
- unsafe extern "C" { … } blocks require the unsafe keyword
- #[unsafe(no_mangle)] replaces #[no_mangle]
- Explicit unsafe { … } required inside unsafe fn bodies
- The compiler is the first reviewer.

Summary — Defense in Depth

FORMAL VERIFICATION
67 Verus proofs · 82 Kani harnesses

E2E SAFETY PROTOCOL · TSN
CRC-32 · sequence tracking · EN 50159 · 802.1Qbv

DETERMINISTIC SCHEDULING
Fixed arena · RTIC integration · LET semantics

SAFE SERIALIZATION
Bounds-checked CDR · no reallocation · Result<T,E>

SAFE TYPES (Rust + C++)
heapless::Vec<T,N> · FixedSequence<T,N> · NROS_TRY

COMPILER ENFORCEMENT
#![no_std] · no alloc · feature gates · Edition 2024

RUST LANGUAGE
Ownership · borrowing · async state machines · Send+Sync